

Методичка 11. Исключения и безопасный ввод

1. Зачем нужна эта тема

Исключения нужны, чтобы программа не падала при ошибках.

Пользователь может ввести не то, что ожидалось:

- вместо числа написать слово;
- оставить пустую строку;
- ввести слишком большое значение;
- указать файл, которого нет;
- попытаться разделить число на ноль.

Без обработки ошибок программа просто остановится и покажет непонятное сообщение.

Исключения позволяют сделать программу устойчивой:

- заметить ошибку;
- не завершать программу аварийно;
- показать понятное сообщение;
- попросить пользователя повторить ввод;
- обработать проблему и продолжить работу.

Эта тема связана с вводом, условиями, циклами и файлами. Особенно часто исключения используются при преобразовании строки в число и при чтении файлов.

2. Главная идея темы

Исключение — это ошибка, которая возникает во время выполнения программы.

Например:

```
number = int("abc")
```

Python не может превратить `"abc"` в число, поэтому возникает ошибка.

Чтобы программа не падала, опасный код помещают в блок `try`.

Если внутри `try` возникает ошибка, программа переходит в блок `except`.

Исключение — ошибка во время выполнения программы.

try — блок, в котором находится код, где может случиться ошибка.

except — блок, который выполняется, если ошибка произошла.

ValueError — ошибка значения, например когда строку `"abc"` пытаются превратить в число.

ZeroDivisionError — ошибка деления на ноль.

FileNotFoundError — ошибка отсутствующего файла.

3. Базовый синтаксис

Простой try except

```
try:
    number = int(input("Введите число: "))
    print(number)
except:
    print("Ошибка: нужно было ввести число")
```

Что происходит:

- программа пытается выполнить код внутри `try`;
- если пользователь ввёл число, всё работает;
- если пользователь ввёл текст, срабатывает `except`.

На что обратить внимание:

- после `try` и `except` ставится двоеточие;
- блоки пишутся с отступами;
- в `try` нужно помещать только тот код, где реально может быть ошибка.

Обработка конкретной ошибки

```
try:
    number = int(input("Введите число: "))
    print(number)
except ValueError:
    print("Ошибка: это не число")
```

Такой вариант лучше, потому что программа ловит именно ошибку преобразования в число.

Деление на ноль

```
try:
    a = int(input("Введите первое число: "))
    b = int(input("Введите второе число: "))

    result = a / b

    print(result)
except ZeroDivisionError:
    print("На ноль делить нельзя")
except ValueError:
    print("Нужно вводить числа")
```

Можно обрабатывать несколько разных ошибок отдельно.

Безопасный ввод через цикл

```
while True:
    try:
        age = int(input("Введите возраст: "))
        break
    except ValueError:
        print("Введите число, а не текст")

print("Возраст:", age)
```

Цикл повторяет ввод, пока пользователь не введёт число.

4. Разбор темы от простого к сложному

Шаг 1. Понять обычную ошибку

```
number = int(input("Введите число: "))
print(number)
```

Если пользователь введёт `abc`, программа упадёт.

Проблема не в `input()`, а в `int()`: строку `abc` нельзя превратить в число.

Шаг 2. Защитить преобразование

```
try:
    number = int(input("Введите число: "))
    print(number)
except ValueError:
    print("Это не число")
```

Теперь программа не падает, а выводит понятное сообщение.

Шаг 3. Повторять ввод до успеха

```
number = None

while number is None:
    try:
        number = int(input("Введите число: "))
    except ValueError:
        print("Ошибка. Введите целое число")

print("Вы ввели:", number)
```

Переменная `number` сначала равна `None`. Когда ввод успешный, в неё записывается число, и цикл останавливается.

Шаг 4. Использовать функцию безопасного ввода

```
def input_int(message):
    while True:
        try:
            number = int(input(message))
            return number
        except ValueError:
            print("Ошибка. Нужно ввести целое число")

age = input_int("Введите возраст: ")
print("Возраст:", age)
```

Функция делает безопасный ввод многоразовым.

Теперь не нужно писать `try except` каждый раз заново.

Шаг 5. Обработать ошибку файла

```
try:
    with open("input.txt", "r", encoding="utf-8") as file:
        text = file.read()

    print(text)
except FileNotFoundError:
    print("Файл не найден")
```

Если файла нет, программа не падает.

Шаг 6. Обработать несколько ошибок

```
try:
    a = int(input("a = "))
    b = int(input("b = "))

    print(a / b)
except ValueError:
    print("Нужно вводить числа")
except ZeroDivisionError:
    print("На ноль делить нельзя")
```

Разные ошибки получают разные сообщения.

Шаг 7. Не прятать все ошибки подряд

```
try:
    number = int(input("Введите число: "))
    print(100 / number)
except ValueError:
    print("Ошибка ввода числа")
except ZeroDivisionError:
    print("Нельзя делить на ноль")
```

Так лучше, чем просто `except:`, потому что понятно, какую ошибку программа обрабатывает.

5. Частые ошибки учеников

Ошибка 1. Не используют try при опасном вводе

Неправильно:

```
age = int(input("Введите возраст: "))
print(age)
```

Почему это ошибка:

Если пользователь введёт текст, программа упадёт.

Правильно:

```
try:
    age = int(input("Введите возраст: "))
    print(age)
except ValueError:
    print("Возраст нужно вводить числом")
```

Ошибка 2. Используют слишком общий except

Неправильно:

```
try:
    number = int(input("Введите число: "))
    print(10 / number)
except:
    print("Ошибка")
```

Почему это плохо:

Непонятно, что именно произошло: неправильный ввод или деление на ноль.

Правильно:

```
try:
    number = int(input("Введите число: "))
    print(10 / number)
except ValueError:
    print("Нужно ввести число")
except ZeroDivisionError:
    print("На ноль делить нельзя")
```

Ошибка 3. Помещают слишком много кода в try

Неправильно:

```
try:
    name = input("Имя: ")
    age = int(input("Возраст: "))
    print("Привет")
    print("Возраст:", age)
    print("Конец программы")
except ValueError:
    print("Ошибка")
```

Почему это не лучший вариант:

В `try` лучше помещать только тот код, где ожидается ошибка.

Правильно:

```
name = input("Имя: ")

try:
    age = int(input("Возраст: "))
except ValueError:
    print("Возраст нужно ввести числом")
else:
    print("Привет")
    print("Возраст:", age)
```

Ошибка 4. После ошибки программа продолжает работать с плохими данными

Неправильно:

```
try:
    age = int(input("Возраст: "))
except ValueError:
    print("Ошибка")

print(age)
```

Почему это ошибка:

Если ввод был неправильным, переменная `age` может не создаться.

Правильно:

```
try:
    age = int(input("Возраст: "))
    print(age)
except ValueError:
    print("Ошибка: возраст должен быть числом")
```

Или безопаснее через функцию с повтором.

Ошибка 5. Забывают повторить ввод

Неправильно:

```
try:
    number = int(input("Введите число: "))
except ValueError:
    print("Ошибка")

print("Программа завершена")
```

Почему это может быть недостаточно:

Программа сообщает об ошибке, но не даёт пользователю исправиться.

Правильно:

```
while True:
    try:
        number = int(input("Введите число: "))
        break
    except ValueError:
        print("Попробуйте ещё раз")

print("Число:", number)
```


Ошибка 6. Ловят не ту ошибку

Неправильно:

```
try:
    number = int("abc")
except ZeroDivisionError:
    print("Деление на ноль")
```

Почему это ошибка:

Здесь возникает `ValueError`, а не `ZeroDivisionError`.

Правильно:

```
try:
    number = int("abc")
except ValueError:
    print("Нельзя преобразовать строку в число")
```

Ошибка 7. Используют исключения вместо обычных условий

Неправильно:

```
try:
    number = int(input("Введите число: "))
    print(10 / number)
except ZeroDivisionError:
    print("На ноль нельзя")
```

Код рабочий, но иногда проверку можно сделать понятнее через условие:

```
number = int(input("Введите число: "))

if number == 0:
    print("На ноль нельзя")
else:
    print(10 / number)
```

Исключения нужны для непредвиденных ситуаций, а простые проверки иногда лучше делать через `if`.

6. Мини-примеры для закрепления

Пример 1. Ввод числа

```
try:
    number = int(input("Введите число: "))
    print(number)
except ValueError:
    print("Это не число")
```

Закрепляется обработка `ValueError` .

Пример 2. Деление

```
try:
    a = int(input("a = "))
    b = int(input("b = "))
    print(a / b)
except ZeroDivisionError:
    print("На ноль делить нельзя")
except ValueError:
    print("Введите числа")
```

Закрепляется несколько `except` .

Пример 3. Безопасный ввод в цикле

```
while True:
    try:
        number = int(input("Введите число: "))
        break
    except ValueError:
        print("Ошибка ввода")

print(number)
```

Закрепляется повторение ввода до правильного значения.

Пример 4. Функция безопасного ввода

```
def input_int(message):
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Введите целое число")

age = input_int("Возраст: ")
print(age)
```

Закрепляется функция и `return`.

Пример 5. Файл не найден

```
try:
    with open("data.txt", "r", encoding="utf-8") as file:
        text = file.read()

    print(text)
except FileNotFoundError:
    print("Файл не найден")
```

Закрепляется обработка файловой ошибки.

Пример 6. Проверка цены

```
try:
    price = int(input("Цена: "))

    if price < 0:
        print("Цена не может быть отрицательной")
    else:
        print("Цена:", price)
except ValueError:
    print("Цена должна быть числом")
```

Закрепляется сочетание `try except` и условий.

Пример 7. Безопасный ввод возраста

```
def input_age():
    while True:
        try:
            age = int(input("Введите возраст: "))

            if age < 0:
                print("Возраст не может быть отрицательным")
            else:
                return age
        except ValueError:
            print("Введите число")

age = input_age()
print("Возраст:", age)
```

Закрепляется проверка типа и логики значения.

7. Практические задания

Лёгкий уровень

1. Написать программу, которая просит число и обрабатывает ошибку, если введён текст.
2. Написать программу, которая делит 100 на введённое число и обрабатывает деление на ноль.
3. Написать программу, которая пытается открыть файл и сообщает, если файла нет.
4. Написать безопасный ввод возраста.
5. Написать безопасный ввод цены товара.

Средний уровень

1. Написать функцию `input_int(message)`, которая повторяет ввод, пока пользователь не введёт целое число.
2. Написать функцию `input_positive_int(message)`, которая принимает только положительное целое число.
3. Пользователь вводит два числа. Обработать ошибки ввода и деления на ноль.
4. Программа читает числа из файла. Если файла нет, вывести понятное сообщение.
5. Пользователь вводит оценку от 1 до 5. Обработать неправильный тип и неправильный диапазон.

Повышенный уровень

1. Сделать меню, где пользователь выбирает команду числом. Если введён текст, программа не падает.
2. Написать функцию безопасного ввода числа в диапазоне от `min_value` до `max_value`.
3. Прочитать числа из файла и посчитать сумму. Если в файле есть строка, которую нельзя превратить в число, вывести сообщение об ошибке.

8. Контрольные вопросы

1. Что такое исключение?
2. Зачем нужны исключения?
3. Что делает блок `try` ?
4. Что делает блок `except` ?
5. Что такое `ValueError` ?
6. Когда возникает `ZeroDivisionError` ?
7. Когда возникает `FileNotFoundError` ?
8. Почему плохо писать просто `except:` без типа ошибки?
9. Почему иногда нужен цикл вместе с `try except` ?
10. Чем проверка через `if` отличается от обработки исключения?
11. Почему опасно использовать переменную после неудачного `try` ?
12. Как сделать функцию безопасного ввода?

9. Краткий конспект в конце

Главная идея темы: исключения помогают программе не падать при ошибках.

Основная конструкция:

```
try:
    опасный_код
except ТипОшибки:
    что_делать_при_ошибке
```

Пример:

```
try:
    number = int(input("Введите число: "))
except ValueError:
    print("Это не число")
```

Важно запомнить:

- `try` содержит код, где может быть ошибка;
- `except` обрабатывает ошибку;
- лучше ловить конкретные ошибки;
- `ValueError` часто возникает при `int(input())`;
- `ZeroDivisionError` возникает при делении на ноль;
- `FileNotFoundError` возникает, если файла нет;
- для безопасного ввода удобно использовать цикл.

Частые ошибки:

- не обрабатывать опасный ввод;
- писать слишком общий `except`;
- помещать в `try` слишком много кода;
- использовать переменную, которая могла не создаться;
- ловить не тот тип ошибки;
- не давать пользователю повторить ввод.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- понимать, что такое исключение;
- использовать `try except`;
- обрабатывать `ValueError`;
- обрабатывать `ZeroDivisionError`;
- обрабатывать `FileNotFoundError`;
- делать безопасный ввод числа;
- повторять ввод до правильного значения;
- писать функцию безопасного ввода;
- отличать обычную проверку через `if` от обработки ошибки;
- находить типичные ошибки в коде с исключениями.